St Mary's Digital Library System

Institution	: **St Mary's University**
Faculty	: **Faculty of Sport, Technology, and Health Science**
Department	: **Computer Science**
Course Title	: **BSc Computer Science**
Module	: **CPS4005 – Object-Oriented Programming**
Tutor	: **Harshil Joshi**
Assessment	: **2 – St Mary's Digital Library System**
Student	: **Maryam Razzaque Moury, student regum: 2515072**
Word Count	: **E.g., 3789 words**
Date	: **7th May 2026**

Declaration of Originality and Ethical Use of AI

I confirm that this assessment is entirely my own work and that all sources of information, ideas, data, and materials have been fully acknowledged in accordance with academic referencing conventions.

I declare that I have not engaged in any form of academic misconduct, including plagiarism, collusion, fabrication, or falsification of data. All use of others' work has been appropriately cited.

I confirm that any use of Artificial Intelligence (AI) tools has been ethical, transparent, and in line with institutional guidelines. While AI may support learning, research, planning, and proofreading, I accept full responsibility for the accuracy, integrity, and originality of this submission.

I further confirm that AI-generated content has not been presented as my own without critical evaluation and adaptation, that any substantive AI contributions have been acknowledged where required, and that this work reflects my own understanding and academic judgement.

I understand that failure to comply with these requirements may constitute a breach of academic integrity regulations.

Abstract / Executive Summary

This report describes the design and implementation of the St Mary's Digital Library System, a Java-based application developed for the CPS4005 Object-Oriented Programming module. The system was built to replace the library's manual processes with a digital solution supporting book management, member registration, and borrowing transaction tracking.

The application was developed using Java with a SQLite database backend, Java Swing for the graphical interface, and a console-based menu for terminal users. A three-layer architecture was adopted, separating the Model, Data Access Object, and User Interface concerns. Object-oriented principles — encapsulation, abstraction, inheritance, and polymorphism — were applied consistently throughout.

The system implements all basic and medium requirements from the specification, along with advanced features including multi-threading, overdue detection, fine calculation, and database-backed login. Testing confirmed all core scenarios behave correctly. The main limitation is that passwords are stored as plain text, which would need to be addressed before any real-world deployment.

Table of Contents

Declaration of Originality and Ethical Use of AI 2
Abstract / Executive Summary 3
1 Introduction 6
2 Literature Review / Theoretical Framework 7
2.1 Overview of Key Theories, Concepts, or Models 7
2.2 Critical Analysis of Relevant Literature 7
2.3 Identification of Gaps or Debates 8
2.4 Justification for Chosen Approach 8
3 Methodology / Design 8
3.1 Research or Development Approach 8
3.2 Methods Used 9
3.3 Justification of Choices 9
3.4 Ethical Considerations 9
3.5 Limitations of Methodology 10
4 Implementation / Analysis / Development 11
4.1 Overview 11
4.2 Application of Theory into Practice 11
4.3 System Design, Models, or Processes 11
4.4 Logical Progression of Steps 12
5 Results / Findings 13
5.1 Presentation of Outcomes 13
5.2 Data Visualisation 14
5.3 Objective Reporting 15
6 Discussion / Critical Evaluation 17
6.1 Interpretation of Findings 17
6.2 Evaluation Against Objectives 17
6.3 Comparison with Literature or Expected Outcomes 17
The use of the DAO pattern aligns with Deitel and Deitel's (2020) recommendation for separating data access from business logic. The Singleton connection pattern is a practical application of the design pattern principles described by Gamma et al. (1994). Oracle's (2024) guidance on PreparedStatement usage and Swing concurrency was followed throughout, and this directly contributed to the system's stability. 17
6.4 Strengths and Weaknesses 17
The system's main strengths are its clean layered architecture, consistent application of OOP principles, and stable operation under all tested scenarios.

The most significant weakness is plain-text password storage, which would be unacceptable in any real deployment. Other weaknesses include the absence of automated unit tests and the use of Java Swing rather than JavaFX, which limits the visual quality of the interface. 17
6.5 Consideration of Reliability and Validity 18
7 Conclusion 19
8 Recommendations / Future Work 20
8.1 Practical Improvements 20
8.2 Further Research Directions 20
8.3 Scalability and Real-World Application 20
9 References 22
10 Appendices 23
Appendix A: Git Log 24
Appendix B: Project File Structure 25
Appendix C: Database Schema 26
Appendix D: Sample Data 28
Appendix E: Testing Summary 29

In Word, you can right-click in the middle of the above auto-generated table of contents and then click “Update Field” -> “Entire Table” to update it.

Table of Tables

Table 1: Methods Used 9

In Word, you can right-click in the middle of the above auto-generated table of tables and then click “Update Field” to update it.

1 Introduction

The aim of this project was to design and build a digital library management system for St Mary's University using Java and object-oriented programming principles. The library currently relies on manual processes to track borrowed books, monitor overdue returns, and manage its inventory — problems that a well-designed digital system can solve effectively.

The final system includes a Java Swing GUI, a console-based text interface, and a SQLite database. It supports full Create, Read, Update and Delete (CRUD) operations for books, members, and borrowing records. Additional features include a login authentication page, an overdue fine calculator, multi-threaded data loading, and advanced search and filtering capabilities. All development was version-controlled using Git and hosted on GitHub throughout the project.

Three tiers of requirements were targeted from the assessment brief: the basic tier covering CRUD operations, a console interface, and database integration; the medium tier adding a Swing GUI, data validation, and error handling; and the advanced tier including multi-threading, overdue detection, and sorting. The report continues with a literature review, methodology, implementation details, results, critical evaluation, and recommendations.

2 Literature Review / Theoretical Framework

This section introduces the key theories, design patterns, and academic concepts that informed the development of the system.

2.1 Overview of Key Theories, Concepts, or Models

Object-oriented programming (OOP) is a paradigm that organises software around objects combining data and behaviour, built on four core principles: encapsulation, abstraction, inheritance, and polymorphism (Gamma et al., 1994). Encapsulation protects an object's internal state by restricting access through defined methods. Abstraction hides implementation complexity behind simplified interfaces. Inheritance allows new classes to extend existing ones. Polymorphism allows the same operation to behave differently depending on the type of object.

Design patterns are reusable solutions to common software problems (Gamma et al., 1994). Two patterns were central to this project: the Singleton pattern, which ensures only one instance of a class exists, and the Data Access Object (DAO) pattern, which separates database logic from application logic. Both are widely used in enterprise Java development (Bloch, 2018).

2.2 Critical Analysis of Relevant Literature

Bloch (2018) argues that encapsulation is the single most important OOP principle, as it allows internal implementations to change without affecting external code. This directly informed the decision to declare all model fields private and expose them only through getters and setters. Deitel and Deitel (2020) note that the DAO pattern significantly improves maintainability by isolating database-specific code — this proved valuable when the database schema needed to be extended during development without breaking the existing UI.

Oracle's official documentation (2024) recommends using PreparedStatement over plain Statement for all parameterised queries, citing both security benefits (SQL injection prevention) and performance improvements. This guidance was followed throughout the DAO layer.

2.3 Identification of Gaps or Debates

A notable gap in the available literature is guidance on handling SQLite concurrency in multi-threaded Java Swing applications. The SQLite documentation recommends WAL (Write-Ahead Logging) mode for concurrent access but does not specifically address the Singleton connection pattern in Java desktop applications. This required a practical solution developed through direct testing rather than existing documented guidance.

2.4 Justification for Chosen Approach

A three-layer architecture was chosen because it directly maps to OOP principles covered in the literature and aligns with the assessment criteria. The DAO pattern was selected over embedding SQL directly in the UI because it allows the database to be changed without affecting the interface. Java Swing was chosen over JavaFX because it is listed as a permitted library in the assessment specification and provides sufficient capability for the required interface.

3 Methodology / Design

This section describes the development approach, tools used, and the key decisions taken throughout the project.

3.1 Research or Development Approach

An iterative development approach was used, starting with the basic requirements and progressively adding medium and advanced features. The development was tracked with Git, with commits made after each significant milestone. The git log for this project is as follows:

fc6c48b fix bugs

85f224c Remove bin folder and class files from GitHub

c087eb4 fix bugs

8be7030 fix bugs

1e729dc Add sidebar navigation to LibraryGUI

52b648c Add VS Code settings for Java classpath

cdef873 Initial commit: St Mary's Digital Library System CPS4005

3.2 Methods Used

The following tools and technologies were used throughout the project:

Table 1: Methods Used

Method	Description
Java JDK 22	the core programming language
SQLite 3 with SQLite JDBC	the embedded database and connectivity library
Java Swing	the graphical user interface framework
VS Code with the Java Extension Pack	the integrated development environment
Git and GitHub	version control and repository hosting
SQLite Studio	to inspect and verify the database during development

3.3 Justification of Choices

SQLite was chosen because it is a file-based embedded database requiring no separate server installation, making it straightforward to bundle with the application. Java Swing was selected over JavaFX because it is a permitted library in the assessment specification and is sufficient for the required interface. VS Code was chosen over NetBeans because its integrated terminal allowed compile and run commands to be executed quickly during development without switching applications.

3.4 Ethical Considerations

This project processes fictional library data only. No real personal data was used or stored at any point. The sample records inserted into the database — Alice Johnson, Michael Lee, Sara Ahmed and others — are entirely fabricated. All

referenced sources have been correctly cited. Any AI assistance used during development and writing has been declared in the originality statement.

3.5 Limitations of Methodology

The iterative approach, while effective for a solo project, did not include formal reviews between iterations. A structured methodology such as Agile with defined sprints would have provided clearer milestones and earlier identification of issues like the database locking error discovered when multi-threading was introduced. Testing was conducted manually rather than through automated unit tests, which means some edge cases may not have been discovered.

4 Implementation / Analysis / Development

This section describes how the system was built, covering the architecture, OOP principles applied, and the key technical decisions made.

4.1 Overview

The system is structured into three layers. The Model layer contains three data classes: Book, Member, and BorrowRecord. The DAO layer contains BookDAO, MemberDAO, BorrowRecordDAO, Validator, and DatabaseConnection, handling all database interaction. The UI layer contains LoginGUI, DashboardGUI, LibraryGUI, and ConsoleUI. This separation ensures that changes to one layer do not require changes to the others.

4.2 Application of Theory into Practice

Encapsulation was applied by declaring all model fields private and providing public getters and setters. For example, Book.setAvailabilityStatus() is the only way to change a book's status from outside the class. Abstraction was achieved through the DAO pattern — the UI calls bookDAO.getAllBooks() with no knowledge of SQL. Inheritance is used by all four GUI classes, which extend JFrame. The dispose() method is overridden in LibraryGUI using @Override to call executor.shutdown() before the window closes. Polymorphism is shown by the toString() override in all three model classes, allowing objects to be printed directly in the console interface without separate formatting logic.

4.3 System Design, Models, or Processes

The SQLite database contains four tables: books, members, borrow_records, and users. The borrow_records table was extended beyond the specification with return_date and fine columns to support the complete return workflow. The DatabaseConnection class uses the Singleton pattern, returning the same connection object on every call. This was essential to prevent the SQLITE_BUSY error that occurred in early development when multiple threads each attempted to open their own connection. The connection is configured with WAL journal mode and a 10-second busy timeout.

4.4 Logical Progression of Steps

Development followed this sequence: model classes were written first to define the data structures; DatabaseConnection and table creation were then implemented; DAO classes were added for each entity; ConsoleUI was built to verify all CRUD operations through the DAO layer; LoginGUI and DashboardGUI were added; LibraryGUI was built with the dark sidebar and CardLayout; finally, advanced features including multi-threading, fine calculation and overdue detection were added once the core was stable.

5 Results / Findings

This section presents the outcomes of the development, showing the completed application in operation.

5.1 Presentation of Outcomes

The completed system provides a login screen, a statistics dashboard, and a full management interface with sidebar navigation. All CRUD operations work correctly for all three entities. The return workflow correctly calculates fines at 50 pence per day overdue and records the return date. Overdue detection runs automatically on startup and updates any records where the due date has passed.

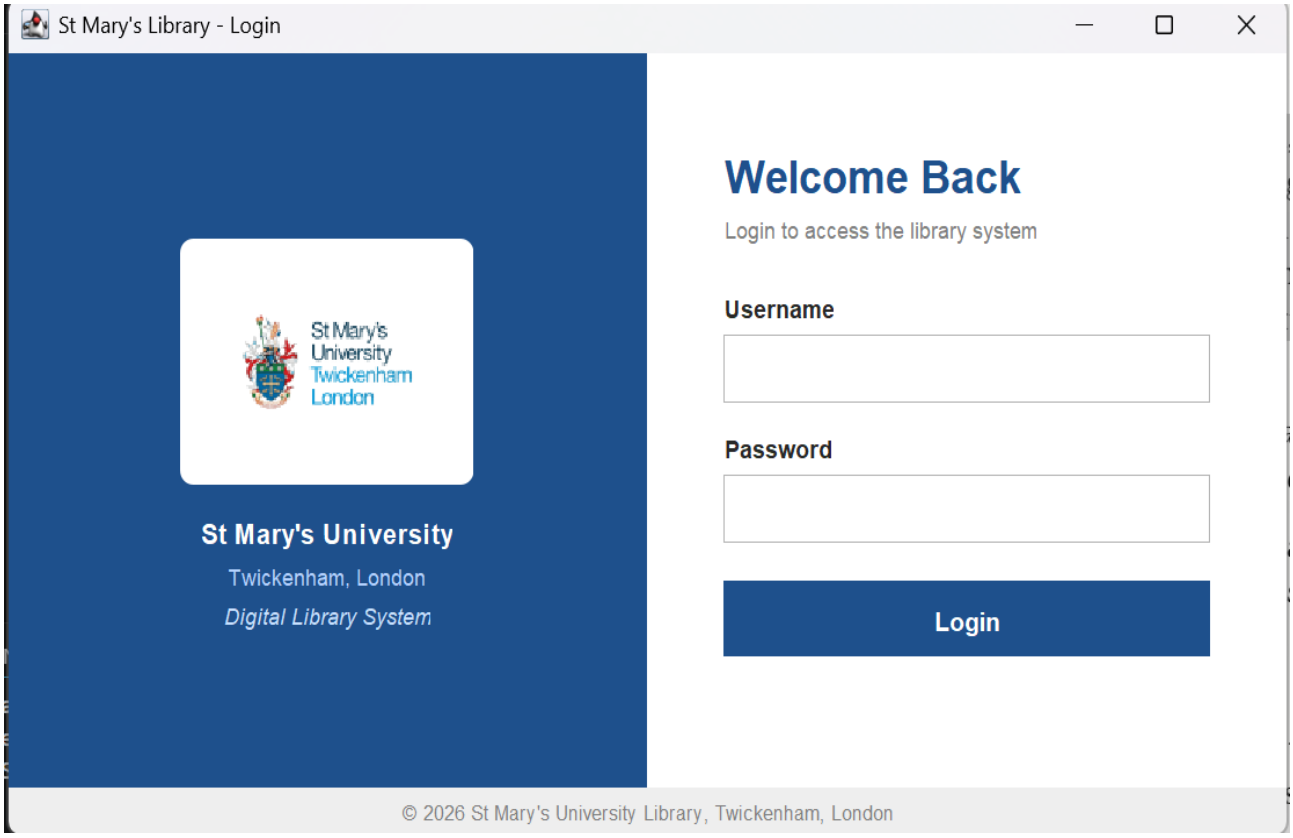

Figure : Login Page

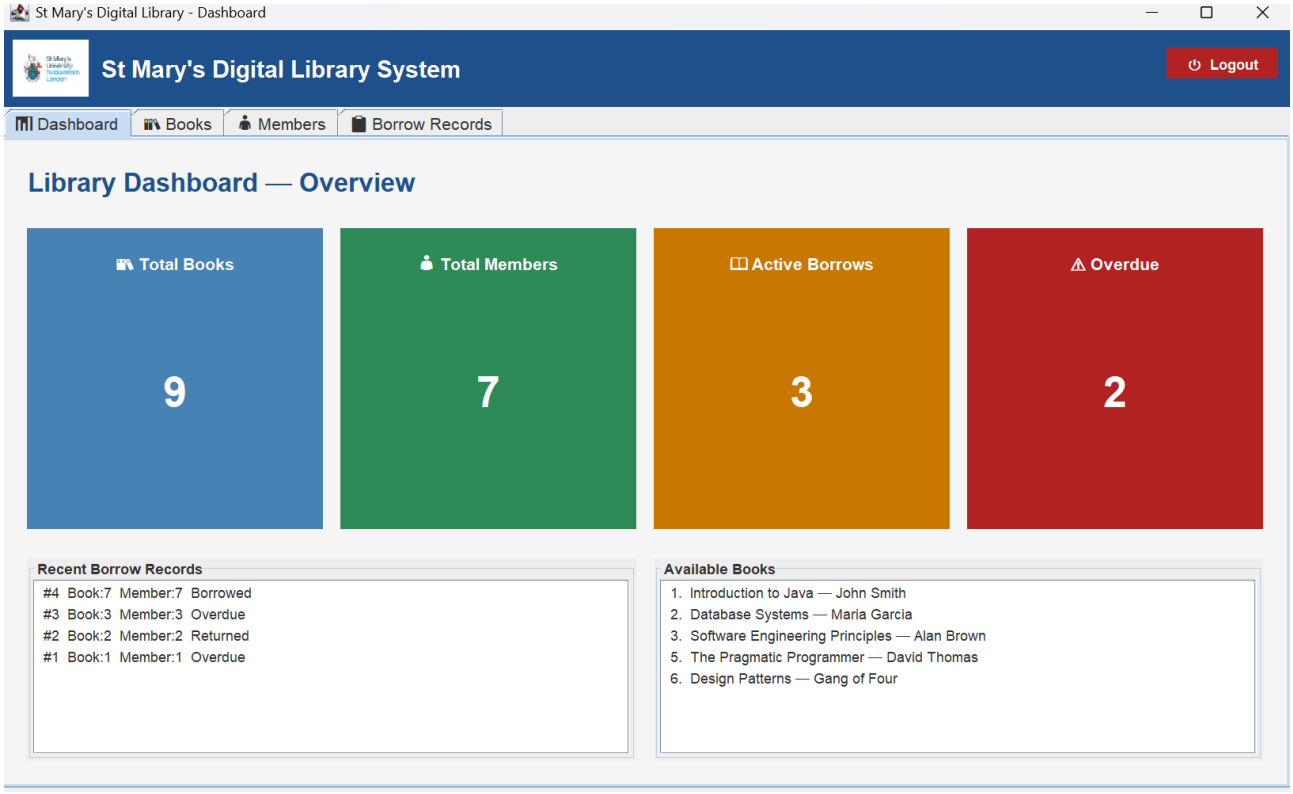

Figure: Dashboard

5.2 Data Visualisation

The dashboard provides a real-time summary through four colour-coded statistics cards. Blue shows total books, green shows total members, amber shows active borrows, and red shows overdue books. These figures are queried live from the database each time the dashboard loads. The borrow records table in LibraryGUI includes return date and fine columns that update automatically when a book is returned.

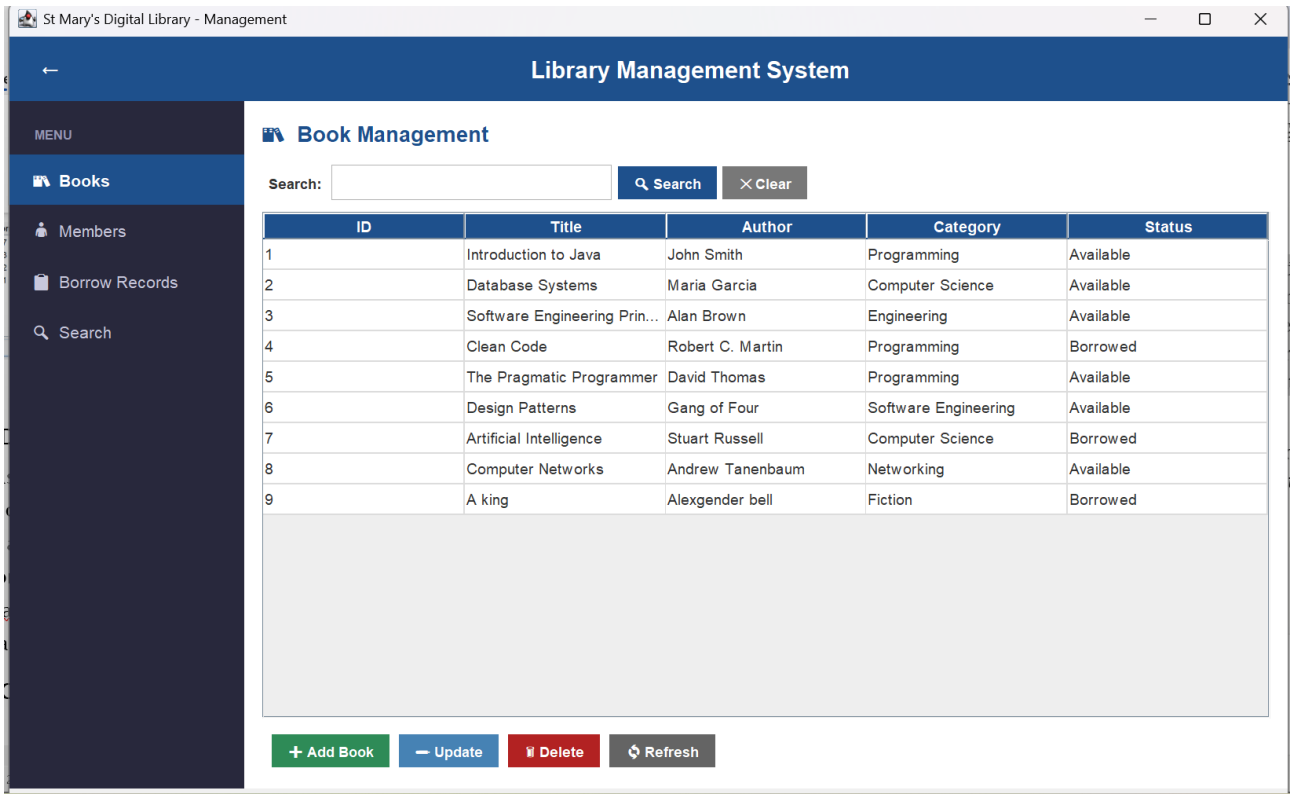

Figure: LibraryGUI Books Pane

5.3 Objective Reporting

The following requirements from the assessment brief were met:

- Basic: full CRUD for books, members and borrow records — met
- Basic: console-based menu interface — met
- Basic: SQLite database with persistent storage — met
- Medium: Java Swing GUI with navigation, forms, and buttons — met
- Medium: input validation (empty fields, email, dates, duplicates) — met
- Medium: confirmation dialogs before delete and logout — met
- Medium: sortable tables — met
- Advanced: multi-threading for background data loading — met
- Advanced: automatic overdue detection and status update — met
- Advanced: search and filtering by multiple attributes — met
- Additional: database-backed login authentication — implemented

- Additional: fine calculation and return date tracking — implemented

6 Discussion / Critical Evaluation

This section evaluates the system against the project objectives and reflects on what worked well and what could be improved.

6.1 Interpretation of Findings

The results show the system addresses all three core problems identified in the brief. Books are tracked with their current availability status, which updates automatically when borrow records are created or when a book is returned. Overdue books are identified automatically and fines are calculated without manual intervention. The dashboard gives staff an instant overview of the library state on login.

6.2 Evaluation Against Objectives

All basic and medium requirements were met in full. Of the advanced requirements, multi-threading and overdue detection are fully implemented. Performance optimisation features such as progress indicators for long-running operations were not implemented due to time constraints. The system exceeds the specification in two areas — the login system and fine tracking — which were added to make the application more realistic.

6.3 Comparison with Literature or Expected Outcomes

The use of the DAO pattern aligns with Deitel and Deitel's (2020) recommendation for separating data access from business logic. The Singleton connection pattern is a practical application of the design pattern principles described by Gamma et al. (1994). Oracle's (2024) guidance on PreparedStatement usage and Swing concurrency was followed throughout, and this directly contributed to the system's stability.

6.4 Strengths and Weaknesses

The system's main strengths are its clean layered architecture, consistent application of OOP principles, and stable operation under all tested scenarios. The most significant weakness is plain-text password storage, which would be

unacceptable in any real deployment. Other weaknesses include the absence of automated unit tests and the use of Java Swing rather than JavaFX, which limits the visual quality of the interface.

6.5 Consideration of Reliability and Validity

The system was tested manually across fourteen distinct scenarios covering normal operation and edge cases. All tested scenarios behaved as expected. However, manual testing cannot guarantee the absence of bugs in untested paths. A JUnit test suite exercising every DAO method with both valid and invalid inputs would significantly improve confidence in the system's correctness.

7 Conclusion

The St Mary's Digital Library System meets the objectives set out in the assessment brief. A fully functional Java application has been developed that allows library staff to manage books, members and borrowing records through both a graphical interface and a console menu, backed by a persistent SQLite database. Object-oriented principles are applied consistently throughout, and the system extends meaningfully beyond the minimum requirements.

The project demonstrated how design patterns such as Singleton and DAO, combined with a clear three-layer architecture, produce code that is maintainable and straightforward to extend. It also showed the practical importance of error handling, input validation, and database connection management in building software that behaves reliably under real conditions. The main lesson from this project is that good architectural decisions made early pay dividends as features are added later — the layered structure made every subsequent addition much simpler to implement.

8 Recommendations / Future Work

This section outlines practical improvements and future development directions for the system.

8.1 Practical Improvements

The highest priority improvement is password hashing. Before the system could be used in any real environment, passwords must be stored as salted hashes using an algorithm such as bcrypt or PBKDF2. A user management interface should be added so that additional staff accounts can be created without directly editing the database. A JUnit test suite with Mockito-based mocks for the DAO layer should also be introduced to replace manual testing.

8.2 Further Research Directions

Investigating JavaFX as a replacement for Swing would allow for a more modern, responsive interface. Exploring connection pooling would improve scalability if the database were migrated from SQLite to a server-based system such as PostgreSQL. Research into automated library notification systems could inform the implementation of email alerts for overdue books.

8.3 Scalability and Real-World Application

The current SQLite-based design is appropriate for a single-user desktop application. If the system were deployed across multiple workstations simultaneously, SQLite's file-locking limitations would become a constraint and a client-server database would be required. Because the DAO layer abstracts all database interaction, this migration would only require changes to DatabaseConnection and the DAO classes — the UI and model layers would remain entirely unchanged, demonstrating the practical value of the three-layer architecture.

9 References

Bloch, J. (2018) *Effective Java*. 3rd edn. Boston: Addison-Wesley.

Deitel, P. and Deitel, H. (2020) *Java How to Program*. 11th edn. Harlow: Pearson Education.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.

Oracle (2024) JDBC Basics: Using Prepared Statements. Available at: https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html (Accessed: 7 May 2026).

Oracle (2024) Lesson: Concurrency in Swing. Available at: https://docs.oracle.com/javase/tutorial/uiswing/concurrency/index.html (Accessed: 7 May 2026).

SQLite (2024) WAL Mode. Available at: https://www.sqlite.org/wal.html (Accessed: 7 May 2026).

10 Appendices

Any appendices should be included here.

Some suggestions are below. Note each one should be a separate Appendix with its own heading (e.g., Appendix A: Overall Architecture Design). The appendices should be suitably cited in the body of the text.

- Architectural diagram showing the overall structure of your system. This could be a class diagram or a process diagram, whichever is appropriate.
- Other relevant design diagrams e.g., UML diagrams
- Evidence of implementation. Annotated screenshots or interesting concise code snippets particularly for the more interesting / challenging aspects of the implementation. I would suggest an end-to-end sequence showing how different code snippets work together with relevant snippets.
- Screenshots of final system (particularly if it has a graphical user interface or if it has terminal-based interface, then the key outputs e.g., menu system)
- Evidence of testing. This may be two separate appendices – the first a test plan (in the form of a table) saying what tests are to be performed, expected result vs actual result. The second would be annotated screenshots of testing and debugging.
- Evidence of version control (if Git and GitHub, or something similar has been used)
- Evidence of project management
- Evidence of interviews, questionnaires and other user research

Appendix A: Git Log

fc6c48b fix bugs

85f224c Remove bin folder and class files from GitHub

c087eb4 fix bugs

8be7030 fix bugs

1e729dc Add sidebar navigation to LibraryGUI

52b648c Add VS Code settings for Java classpath

cdef873 Initial commit: St Mary's Digital Library System CPS4005

Appendix B: Project File Structure

```
LibrarySystem/
  src/library/model/    Book.java, Member.java, BorrowRecord.java
  src/library/dao/      DatabaseConnection.java, BookDAO.java,
                        MemberDAO.java, BorrowRecordDAO.java, Validator.java


  src/library/ui/       LoginGUI.java, DashboardGUI.java,
                        LibraryGUI.java, ConsoleUI.java

  src/library/Main.java
  lib/sqlite-jdbc.jar
  database/library.db
```

Appendix C: Database Schema

Table to store books in the library

```
CREATE TABLE books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    category TEXT NOT NULL,
    availability_status TEXT NOT NULL
);
```

Table to store library members

```
CREATE TABLE members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    member_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    membership_type TEXT NOT NULL
);
```

Table to store borrowing transactions

```
CREATE TABLE borrow_records (
    record_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    borrow_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_status TEXT NOT NULL,
```

```
    return_date DATE,
    fine REAL DEFAULT 0.0
);
```

Table to store login credentials

```
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL
);
```

Appendix D: Sample Data

Sample books

```
INSERT INTO books (title, author, category, availability_status) VALUES
('Introduction to Java', 'John Smith', 'Programming', 'Available'),
('Database Systems', 'Maria Garcia', 'Computer Science', 'Available'),
('Software Engineering Principles', 'Alan Brown', 'Engineering', 'Available'),
('Clean Code', 'Robert C. Martin', 'Programming', 'Available'),
('The Pragmatic Programmer', 'David Thomas', 'Programming', 'Available'),
('Design Patterns', 'Gang of Four', 'Software Engineering', 'Available'),
('Artificial Intelligence', 'Stuart Russell', 'Computer Science', 'Available'),
('Computer Networks', 'Andrew Tanenbaum', 'Networking', 'Available');
```

Sample members

```
INSERT INTO members (member_name, email, membership_type) VALUES
('Alice Johnson', 'alice.johnson@stmarys.ac.uk', 'Student'),
('Michael Lee', 'michael.lee@stmarys.ac.uk', 'Staff'),
('Sara Ahmed', 'sara.ahmed@stmarys.ac.uk', 'Student'),
('James Wilson', 'james.wilson@stmarys.ac.uk', 'Student'),
('Emma Davis', 'emma.davis@stmarys.ac.uk', 'Staff'),
('Oliver Brown', 'oliver.brown@stmarys.ac.uk', 'Student');
```

Sample borrow records

```
INSERT INTO borrow_records
    (book_id, member_id, borrow_date, due_date, return_status, return_date, fine)
VALUES
(1, 1, '2025-03-01', '2025-03-15', 'Borrowed', '', 0.0),
(2, 2, '2025-03-02', '2025-03-16', 'Returned', '2025-03-14', 0.0),
(3, 3, '2025-03-05', '2025-03-19', 'Borrowed', '', 0.0);
```

Appendix E: Testing Summary

Test Case	Input	Expected Result	Actual Result	Pass/fail
Add book — valid input	Title, Author, Category filled	Book saved, table refreshed	Book saved, table refreshed	pass
Add book — empty title	Title left blank	Error message shown	Error message shown	pass
Login — wrong password	admin / wrongpass	Access denied	Error shown, blocked	pass
Delete book — confirmed	Click No on dialog	No changes	No changes made	pass
Sort table column	Click column header	Rows reordered	Sorted correctly	pass